

Модульное тестирование (Unit Testing)

Стоимость
правок

**Модульные тесты позволяют
исправить ошибки на ранних этапах
разработки** и сэкономить время и
усилия, которые могли бы быть
потрачены на поиск и исправление
проблем в уже сложившемся коде;



Сбор
требований

Проектирование
и разработка

Юнит-
тестирование

Функциональное/
системное
тестирование

Пользовательское
приемочное
тестирование

После
выпуска
приложения

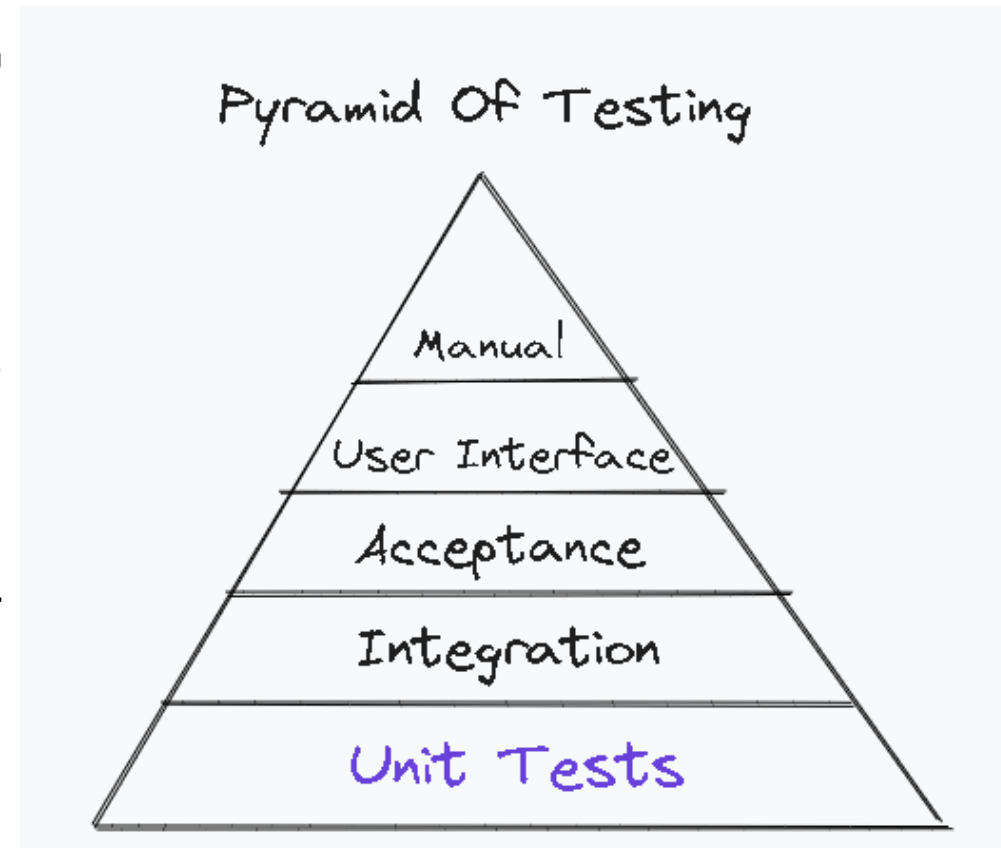
Точка, на которой
найден баг



Модульное тестирование (Unit Testing) – это тип тестирования программного обеспечения, при котором тестируются отдельные модули или компоненты программного обеспечения.

Цель модульного тестирования – проверить, что каждая единица программного кода работает должным образом.

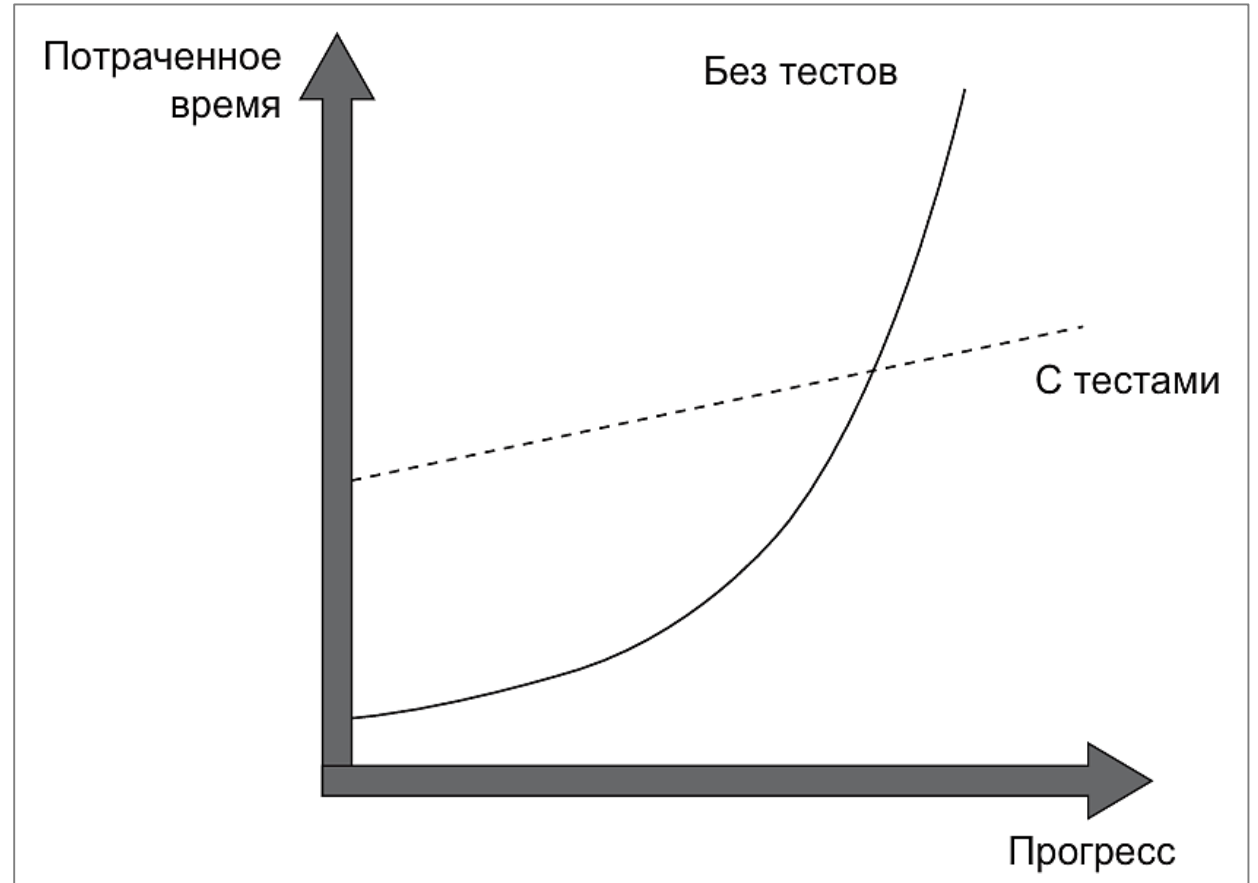
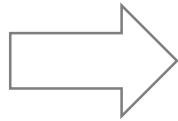
- Модульное тестирование это метод тестирования WhiteBox, который обычно **выполняется разработчиками** на этапе кодирования приложения.
- **Единицей для измерения может служить** отдельная функция, метод, процедура, модуль или объект.
- Юнит-тесты **изолируют часть кода** и проверяют его работоспособность.
- Юнит-тесты находятся на самом базовом уровне жизненного цикла тестирования.



При разработке приложений практически каждый проект включает какое-то количество юнит-тестов. **Соотношение между рабочим и тестовым кодом обычно лежит в диапазоне от 1:1 до 1:3** (на каждую строку рабочего кода приходится от одной до трех строк тестового кода). Иногда это соотношение м.б. до 1:10.

Цель юнит-тестирования — обеспечение стабильного роста программного проекта.

На рисунке изображена динамика роста типичного проекта без тестов. Все начинается быстро, потому что ничего не тормозит. Еще не приняты неудачные архитектурные решения; еще нет существующего кода, который необходимо прорабатывать и поддерживать. Однако с течением времени, когда вы что-то изменяете в коде проекта, увеличивается степень беспорядка в нем — его энтропия. Если не принять должных мер (например, постоянной чистки и рефакторинга), код постепенно усложняется и дезорганизуется.



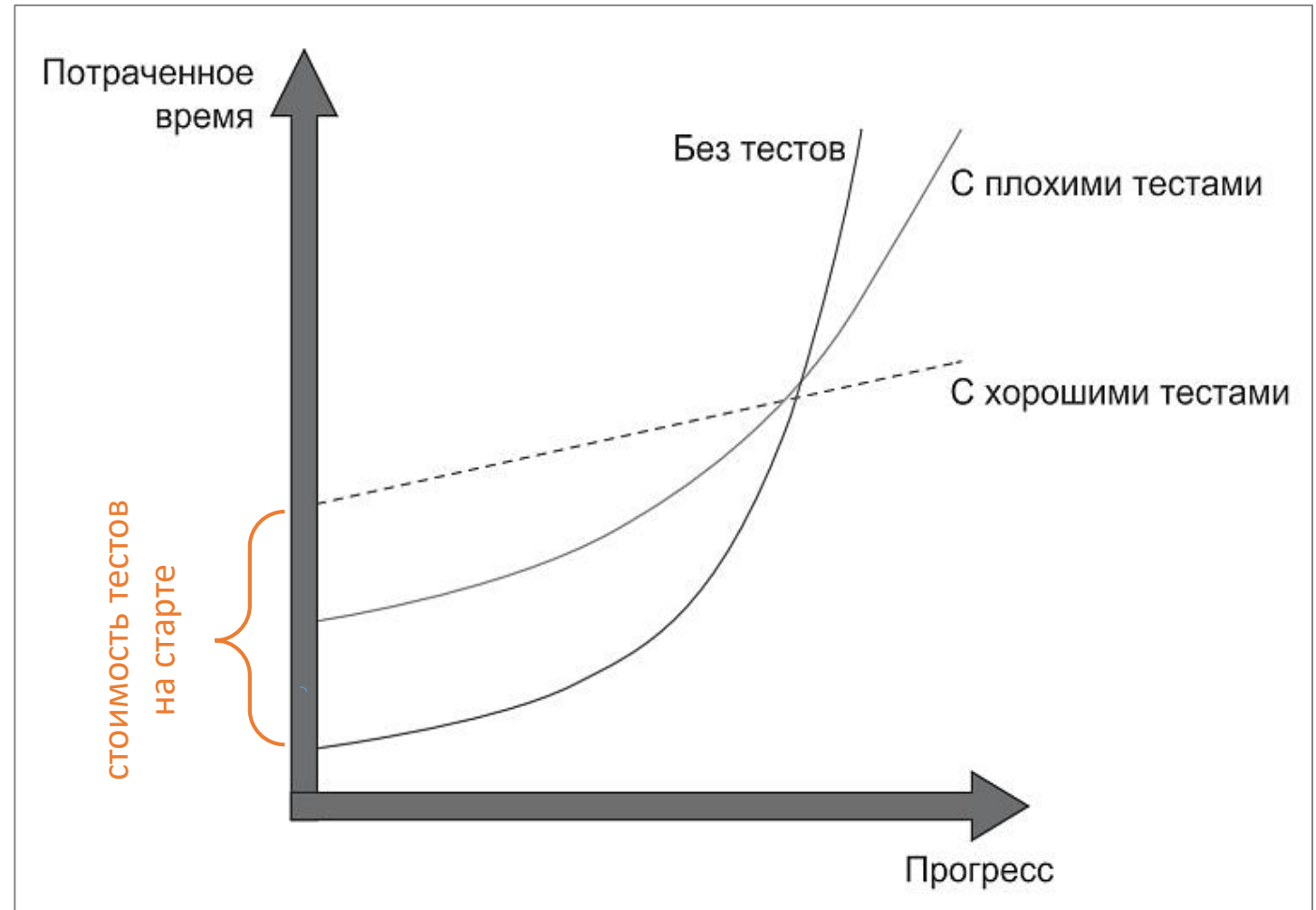
Изображение из книги Владимира Хорикова «Принципы юнит-тестирования»

Невозможно добиться цели юнит-тестирования, просто добавив в проект больше тестов.

Необходимо учитывать как пользу этих тестов, так и затраты на их сопровождение.

Составляющая затрат на сопровождение определяется количеством времени, ушедшего на:

- рефакторинг теста при рефакторинге тестируемого кода;
- выполнение теста при каждом изменении кода;
- отвлечение на ложные срабатывания теста;
- затраты на чтение теста при попытке понять, как работает тестируемый код.

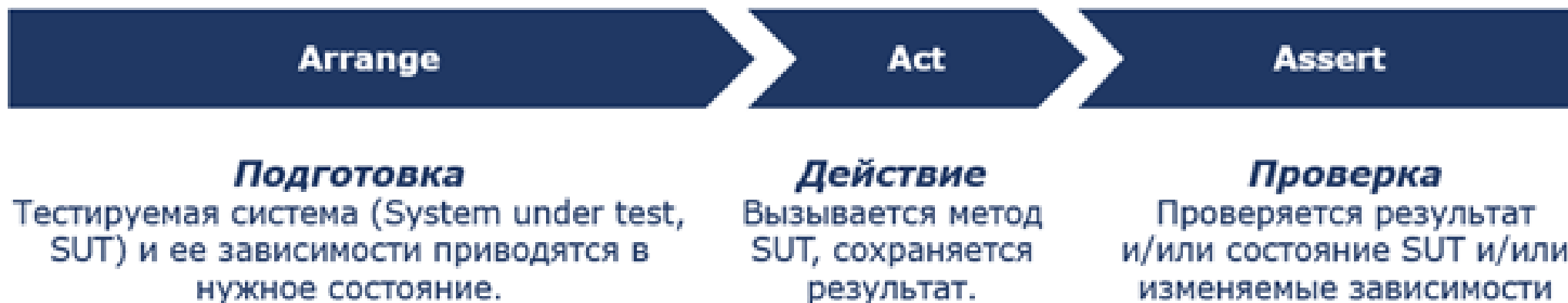


Недостаток юнит-тестирования заключается в том, что тесты требуют начальных вложений, и иногда весьма значительных. Но в долгосрочной перспективе они окупаются, позволяя проекту расти на более поздних стадиях.

ИТАК, хорошие модульные тесты:

- ✓ **позволяют исправить ошибки на ранних этапах разработки** и сэкономить время и усилия, которые могли бы быть потрачены на поиск и исправление проблем в уже сложившемся коде;
- ✓ **помогают предотвратить появление новых ошибок** при внесении изменений в код в будущем;
- ✓ **способствуют повышению надежности кода.** Хороший набор тестов гарантирует, что код будет работать стабильно и предсказуемо даже в сложных ситуациях;
- ✓ **служат документацией к коду.** Описание ожидаемого поведения кода в явном виде делает код более понятным и облегчает его использование и поддержку для других разработчиков;
- ✓ **юнит-тесты позволяют проводить рефакторинг кода без опасений,** что существующая функциональность будет сломана.

Структура юнит теста. Паттерн ААА



Arrange (Настройка): Создать тестовое окружение. Подготовить все необходимое для выполнения теста.

Act (Выполнение): Выполнить тестовый код. Самый маленький блок по объему, чаще всего одна строка кода – вызов метода.

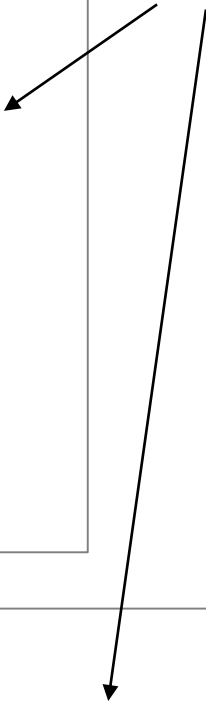
Assert (Проверка): В этом блоке выполняется серия проверок – например, результат сравнивается с ожидаемым. Или проверяется, какие зависимости вызывались, в какой последовательности и какими параметрами. Этот блок отвечает на вопрос, правильно ли работает тестируемая система и фактически заключает в себе ценность теста.

```
void test_function(void)
{
    /* Arrange */
    int expected = 10;
    int actual = 0;
    /* Act */
    actual = add(1,9);
    /* Assert */
    assert(actual, expected);
}
```

class **CalculatorTests**

```
{  
    public void Sum_2Plus5_7Returned()  
    {  
        // arrange  
        var calc = new Calculator();  
        // act  
        var res = calc.Sum(2,5);  
        // assert  
        Assert.AreEqual(7, res);  
    }  
}
```

Можно писать тесты по-разному,
главное придерживаться одного
стиля.



class **CalculatorTests**

```
{  
    public void Sum_2Plus5_7Returned()  
    {  
        Assert.AreEqual(7, new Calculator().sum(2,5));  
    }  
}
```

Assert.AreEqual – это метод, для
проверки равенства двух значений
(NUnit, MSTest и др. фреймворки).

Юнит-тест обладает тремя важными атрибутами:

Юнит-тест должен:

- ✓ проверять правильность работы одного небольшого фрагмента кода – юнита;
- ✓ делать это быстро;
- ✓ **поддерживать изоляцию от другого кода.**

Вопрос изоляции — это корень различий между классической и лондонской школой юнит-тестирования.

Классический стиль написания юнит-теста

пример из книги Владимира Хорикова
«Принципы юнит-тестирования»

```
public enum Product
{
    Shampoo,
    Book
}
```

Допустим, есть интернет-магазин с одним сценарием использования: покупка товара. Если количества товара на складе достаточно, покупка считается успешной, а запас товара сокращается на величину заказа. Если товара недостаточно, то покупка отклоняется, и состояние склада не меняется.

```
[Fact]
public void Purchase_succeeds_when_enough_inventory()
{
    // Arrange
    var store = new Store();
    store.AddInventory(Product.Shampoo, 10);
    var customer = new Customer();

    // Act
    bool success = customer.Purchase(store, Product.Shampoo, 5);

    // Assert
    Assert.True(success);
    Assert.Equal(5, store.GetInventory(Product.Shampoo));
}
```

В этом примере **Customer** – это тестируемая система (SUT, system under test), а **Store** — коллаборатор.

Здесь тест фактически проверяет оба класса (Customer и Store), а не только SUT (Customer).
Такие тесты упадут при любой ошибке в Store, затрагивающей Customer, даже если сам Customer при этом работает правильно. Тесты не изолируют эти два класса друг от друга

Уменьшает количество
товара на складе на 5

Перепишем пример в ЛОНДОНСКОМ СТИЛЕ.

В фазе подготовки (arrange) тесты уже не создают полнофункциональный экземпляр Store; вместо этого для него создается замена при помощи класса Mock из библиотеки Moq.

Также вместо того, чтобы изменять состояние Store добавлением в него товара для дальнейшей покупки, мы напрямую сообщаем моку, как следует реагировать на вызовы HasEnough Inventory(). Мок реагирует на этот запрос так, как требуется тесту, независимо от фактического состояния Store.

```
[Fact]
public void Purchase_succeeds_when_enough_inventory()
{
    // Arrange
    var storeMock = new Mock<IStore>();
    storeMock
        .Setup(x => x.HasEnoughInventory(Product.Shampoo, 5))
        .Returns(true);
    var customer = new Customer();

    // Act
    bool success = customer.Purchase(
        storeMock.Object, Product.Shampoo, 5);

    // Assert
    Assert.True(success);
    storeMock.Verify(
        x => x.RemoveInventory(Product.Shampoo, 5),
        Times.Once);
}
```

Фаза проверки тоже изменилась, и именно здесь кроется ключевое различие. Мы проверяем результат работы метода customer.Purchase, как и прежде, но взаимодействие между Customer и Store теперь проверяется по-другому. Ранее для этого использовалось состояние магазина. Теперь же мы анализируем взаимодействия между Customer и Store: тесты проверяют, какой метод и с какими параметрами Customer вызвал у Store. Для этого в мок передается вызываемый метод (x.RemoveInventory), а также сколько раз этот метод должен был вызываться в течение теста. Если покупка завершается успехом, метод должен вызываться один раз (Times.Once). В случае же неудачи метод не должен вызываться вообще (Times.Never)

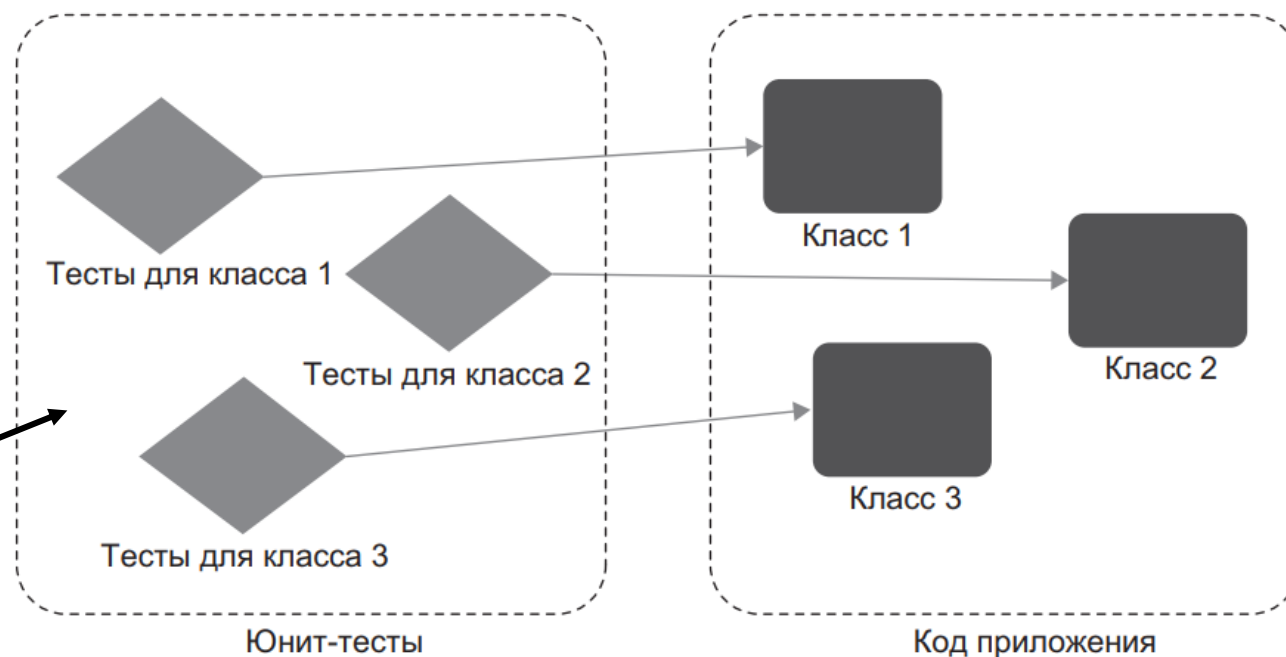
Что означает «изоляция кода» в юнит-тестировании?

Разработчик может изолировать единицу кода для более качественного тестирования. Эта практика подразумевает копирование кода в собственную среду тестирования. *Изоляция кода помогает выявить ненужные зависимости между тестируемым кодом и другими модулями или пространствами данных в продукте.*

Если класс имеет зависимость от другого класса или нескольких классов, все такие зависимости должны быть заменены на тестовые заглушки (test doubles). Это позволяет сосредоточиться исключительно на тестируемом классе, изолировав его поведение от внешнего влияния.

Тестовая заглушка (test double) — объект, который выглядит и ведет себя как его рабочий аналог, но в действительности представляет собой упрощенную версию, более удобную для тестирования.

Изоляции юнит-тестов позволяет установить простые правила тестирования рабочего кода: одному классу соответствует один тест.



На схеме изображена структура **Unit-теста**, разделенная на три классические фазы (шаблон **AAA** — Arrange, Act, Assert).

Основные компоненты и этапы:

SUT (System Under Test): Объект или компонент, который вы непосредственно тестируете.

Mocked Dependency: Внешние зависимости (БД, API), которые заменяются «заглушками» (моками), чтобы изолировать SUT.

Фазы выполнения:

GIVEN (Arrange / Подготовка):

Создание экземпляра SUT через **Constructor**.

Передача моков зависимостей (**Mocks**).

Установка начального состояния через свойства (**Set Property**).

WHEN (Act / Действие):

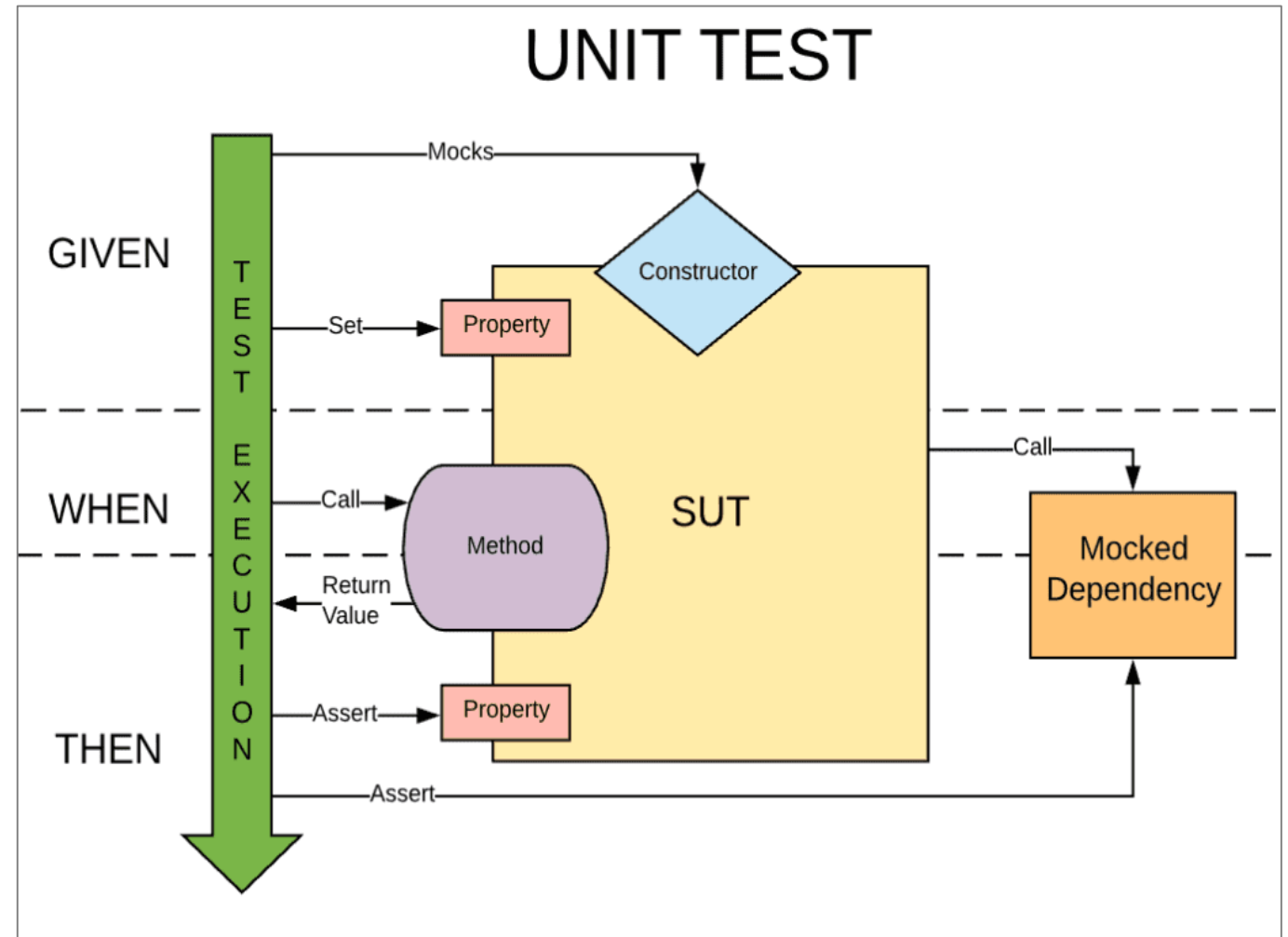
Вызов конкретного метода (**Call Method**), работу которого нужно проверить.

Метод может обращаться к мокам (**Call**) и возвращать результат (**Return Value**).

THEN (Assert / Проверка):

Проверка изменившихся свойств объекта (**Assert Property**).

Проверка того, как SUT взаимодействовал с зависимостями (например, был ли вызван метод у мока).



Тестовые заглушки (тестовые двойники)

Пустышка (Dummy). Такой двойник не содержит поведения и используется в качестве заполнителя параметров. Никогда реально не вызывается.

Подделка (Fake). Это реально написанная реализация, которая имеет более простую логику, чем реальный аналог.



Есть ещё три вида похожих между собой тестовых двойников. Но есть и различия.

Заглушка (Stub) - имеет заранее подготовленные ответы на вызовы методов.

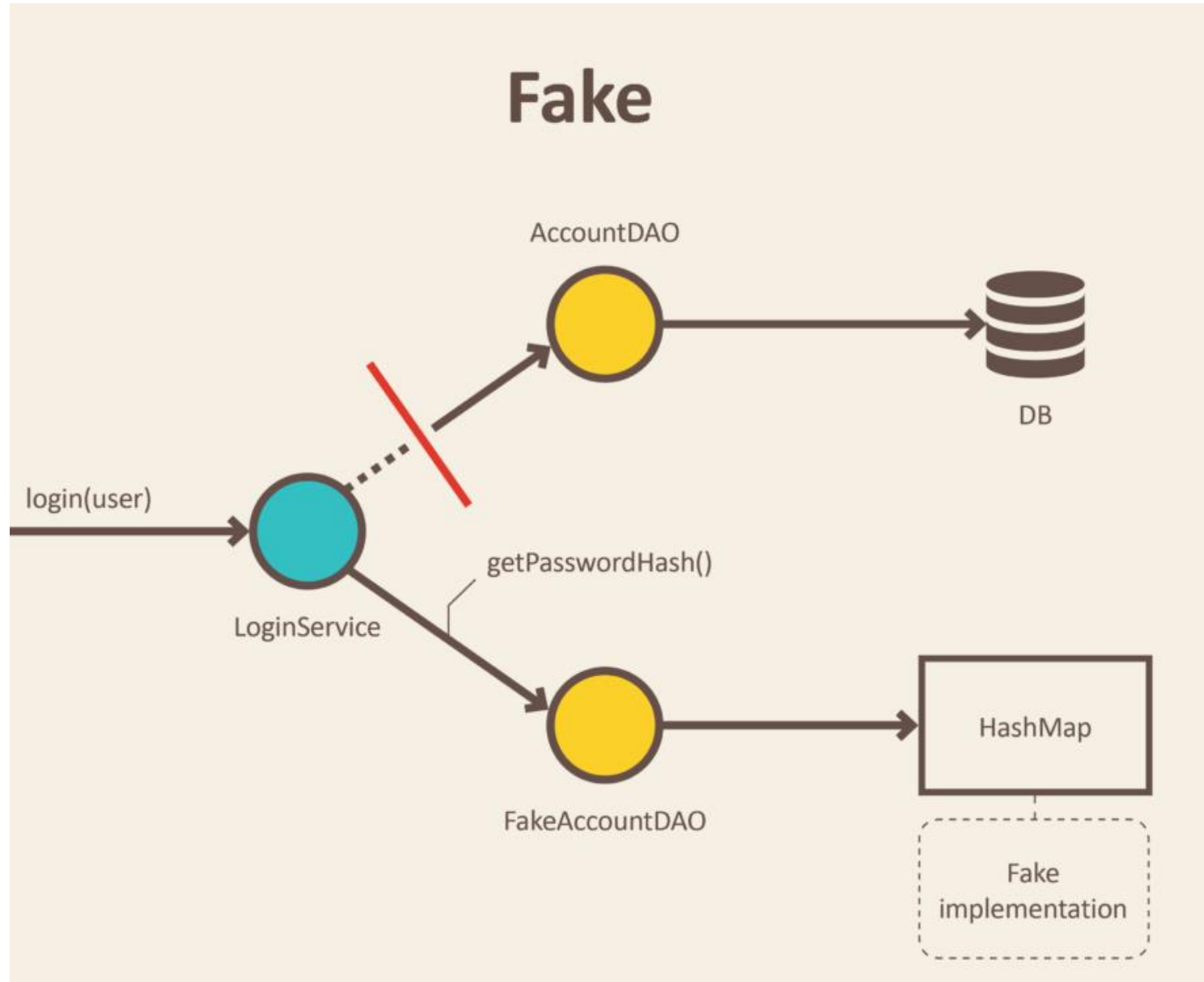
Мок (Mock). Самый сложный из тестовых двойников, но дающий наибольший контроль. Может иметь сложную логику ответов, зависящих от параметров вызовов, их количества, генерировать исключения при вызове методов с неопределенным поведением и имеет другой, полезный для тестирования функционал.

Шпион (Spy). Более сложная система. Это гибрид реального объекта и мока. Он имеет поведение реального объекта, но может записывать определенную информацию о вызове его методов. Также можно переопределить поведение некоторых методов.

Fake

Имитация (или поддельные объекты) — это объекты, имеющие рабочие реализации, но не такие, как у настоящих рабочих объектов. Обычно они имеют упрощённую версию реального объекта.

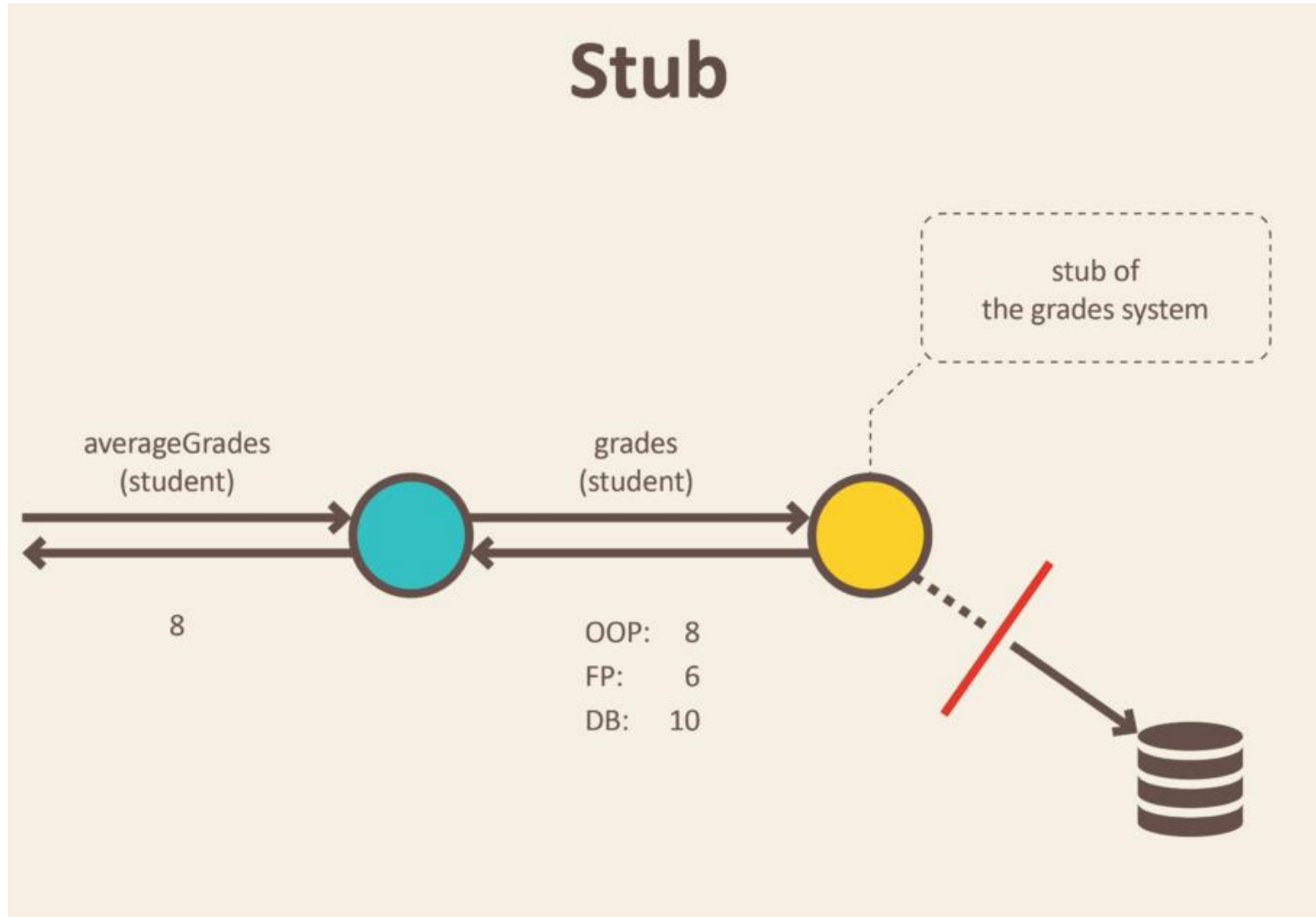
В качестве примера может быть реализация в оперативной памяти объектов доступа к данным (Data Access Object) или репозиторий (Repository). Реализация поддельных объектов не будет привлекать базу данных, но будет использовать простую коллекцию для хранения данных. Это позволяет нам выполнять интеграционный тест сервисов без участия базы данных и выполнения тем самым трудоёмких запросов.



Stub (заглушка)

Заглушка— объект, содержащий predetermined данные и использует их для ответа на вызовы во время тестов. Она используется, когда мы не можем или не хотим привлекать объекты, которые бы отвечали реальными данными или имели бы нежелательные побочные эффекты.

В качестве примера может быть объект, который должен получить некоторые данные из базы данных в качестве результата при вызове метода. Вместо реального объекта, мы вводим заглушку и определяем в ней, какие данные она должна вернуть.



Как измерить покрытие кода тестами?

Для этого есть две чаще всего используемые метрики – процент покрытия строк (code coverage) и процент покрытия логических ветвей (branch coverage) кода.

$$\text{Code coverage} = \frac{\text{Количество выполненных тестами строк кода}}{\text{Общее количество строк кода}}$$

$$\text{Branch coverage} = \frac{\text{Количество выполненных тестами ветвей}}{\text{Общее количество ветвей}}$$

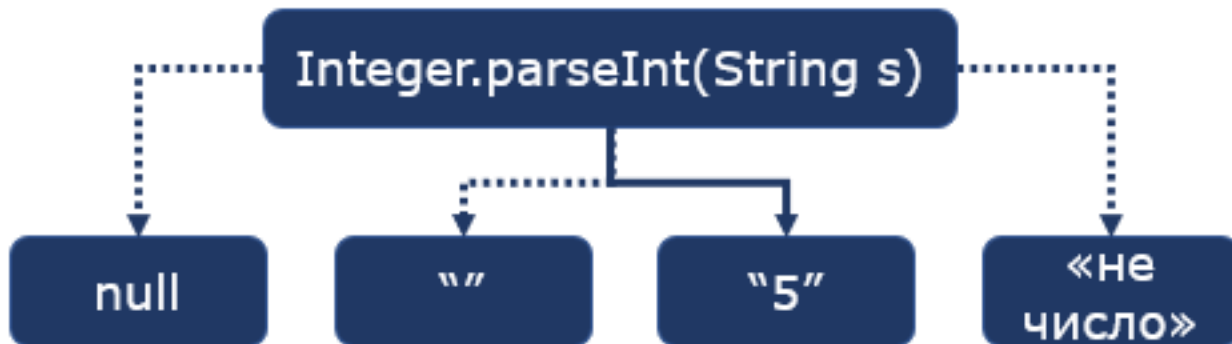
Но даже стопроцентное покрытие тестами не гарантирует хорошего качества тестов. И вот почему. Эти метрики не отражают некоторые важные детали, такие как наличие и полнота проверок (asserts).

Пример

```
public static Integer parse(String numberAsString){  
    return Integer.parseInt(numberAsString);  
}
```

@Test

```
void parseTest() {  
    assertEquals(Integer.valueOf(5), parse("5"));  
}
```



Есть метод, который принимает строку и превращает её в число. Очень простой метод. На него написан простой тест, который передаёт строку со значением «5» и сравнивает число 5 с тем, какое число возвращает метод.

Если мы посмотрим на этот тест и попробуем оценить метрики покрытия, мы увидим, что для этих метрик покрытие будет 100%.

Но, к сожалению, если мы заглянем в документацию метода **parseInt**, мы увидим, что на самом деле мы проверили только один из четырех вариантов. Поэтому с одной стороны у нас покрытие 100%, но с другой стороны мы покрыли по факту только 25% кода.

✓ Избегайте команд if в тестах

Наличие в тестах команды if также является антипаттерном.

Тест (неважно, юнит- или интеграционный) должен представлять собой простую последовательность шагов без ветвлений.

Присутствие команды if означает, что тест проверяет слишком много всего. Следовательно, такой тест должен быть разбит на несколько тестов. Ветвление затрудняют чтение и понимание тестов.



✓ Снижайте количество тест-кейсов для юнита

В среднем на юнит должно приходиться **от 1 до 9 тест-кейсов**. Это очень хороший индикатор качества юнита - если тест-кейсов больше или хочется их сгруппировать, нам точно нужно разделить его на два и больше независимых юнитов.

✓ Избегайте секций Act больше, чем из одного вызова

Если для теста требуется вызвать сначала один метод тестируемой системы, потом другой, высока вероятность того, что API тестируемой системы спроектировано неправильно.

F.I.R.S.T. в тестировании — это акроним, описывающий принципы создания качественных, эффективных и поддерживаемых автоматизированных модульных тестов.

Быстрота (Fast)

Тесты должны выполняться очень быстро. Время выполнения, включая настройку, сам тест и завершение, должно составлять миллисекунды, так как в проекте может быть тысячи тестов.

Изоляция (Isolated/Independent)

Каждый тест должен быть независим. Он должен следовать модели "подготовка, действие, проверка" (Arrange, Act, Assert) без зависимости от других тестов или внешнего окружения.

Повторяемость (Repeatable)

Тесты должны давать одинаковые результаты в любой среде и в любое время, независимо от внешних условий, таких как дата/время или случайные значения.

Самодостаточность (Self-Validating)

Результаты теста должны быть ясны без внешних проверок — тест либо проходит, либо нет, без всякой необходимости в дополнительной интерпретации.

Тщательность (Thorough/Timely)

Тесты должны охватывать все возможные сценарии использования и граничные условия, не ограничиваясь простым стремлением к 100% покрытию кода. Важно тестировать различные объемы данных, безопасность, и корректную обработку исключений.

Быстрота (Fast)

Этот тест медленный, потому что делает реальный запрос к API.

```
// ❌ Плохой тест, который загружает данные из API
it('fetches and displays user data', async () => {
  render(<UserProfile userId="123" />);
  await waitFor(() => expect(screen.getByText(/Username/)).toBeInTheDocument());
});
```

```
// ✅ Использование моков для ускорения тестов,
// чтобы сделать тест быстрым и независимым
jest.mock('api/userApi');
it('displays user data from mock', () => {
  userApi.getUser.mockResolvedValue({ id: '123', name: 'John Doe' });
  render(<UserProfile userId="123" />);
  expect(screen.getByText(/John Doe/)).toBeInTheDocument();
});
```

Больше примеров на соответствие принципам F.I.R.S.T.: <https://habr.com/ru/articles/830962/>

Для упрощения написания и запуска тестов существуют специальные фреймворки для разных языков программирования, например:

- ✓ **unittest**: встроенный модуль для тестирования в языке программирования Python. Он предоставляет широкие возможности и включает в себя возможности для создания фейковых объектов (mock objects).
- ✓ **pytest**: популярный фреймворк для тестирования в Python. Обладает простым и понятным синтаксисом и предоставляет множество функций для удобного написания и выполнения тестов.
- ✓ **JUnit**: фреймворк для тестирования в языке Java. Предоставляет средства для написания и запуска юнит-тестов и широко используется в разработке на Java.
- ✓ **NUnit**: фреймворк для тестирования в языке C#. Предоставляет возможности для создания и запуска тестов в C# и обладает богатым функционалом.
- ✓ **Mocha**: фреймворк для тестирования в JavaScript. Позволяет писать тесты с использованием синтаксиса, близкого к естественному языку, и обладает широкой поддержкой различных типов тестов.

Пример использования моков Python

<https://www.youtube.com/watch?v=wbUgJHiaokY>

```
import requests

def get_joke(): 4 usages
    url='https://api.chucknorris.io/jokes/random'
    response=requests.get(url)

    if response.status_code==200:
        joke=response.json()['value']
    else:
        joke="No jokes"

    return joke

def len_joke(): 2 usages
    joke=get_joke()
    return len(joke)

if __name__=='__main__':
    print(get_joke())
```

В модуле реализованы две функции

Первая **get_joke()** – посылает запрос по url-ссылке и из полученного ответа сохраняет шутку в переменной joke.

Для данной функции нужно написать минимум два юнит теста, для покрытия каждой ветви функции

Вторая функция **len_joke()** возвращает длину шутки из первой функции.

Здесь достаточно одного юнит теста, который проверит как правильно функция считает длину строки

```
3  import unittest
4  from unittest.mock import patch
5  from unittest.mock import MagicMock
6  from Jokes import len_joke, get_joke
7
8  class TestJoke(unittest.TestCase):
9      # указываем путь к объекту который хотим замочать
10     # декоратор patch создает фиктивный объект класса MagicMock()
11     # и передает ссылку на него в декорируемый объект
12     @patch('Jokes.get_joke')
13     def test_len_joke(self, mock_get_joke):
14         mock_get_joke.return_value = 'ahaha'
15         self.assertEqual(len_joke(), second: 5)
```

Класс с
тестами

Юнит-тест
для функции
len_joke()

Для проверки работы функции **len_joke()** нужно знать какую строку вернет функция **get_joke()**, но **get_joke()** возвращает рандомную строку, поэтому вместо вызова **get_joke()** мы обратимся к тестовой заглушке с заранее известной строкой, длина которой будет нам известна.

```
@patch('Jokes.requests')
def test_get_joke(self, mock_requests):
    mock_respons=MagicMock()
    mock_respons.status_code=200
    mock_respons.json.return_value={
        'value': 'Hello world'
    }
    mock_requests.get.return_value=mock_respons
    self.assertEqual(get_joke(), second: 'Hello world')
```

```
@patch('main.requests')
def test_fail_get_joke(self, mock_requests):
    mock_response = MagicMock(status_code=403)
    mock_response.json.return_value = {
        'value': 'No jokes'
    }
    mock_requests.get.return_value = mock_response
    self.assertEqual(get_joke(), 'No jokes')
```